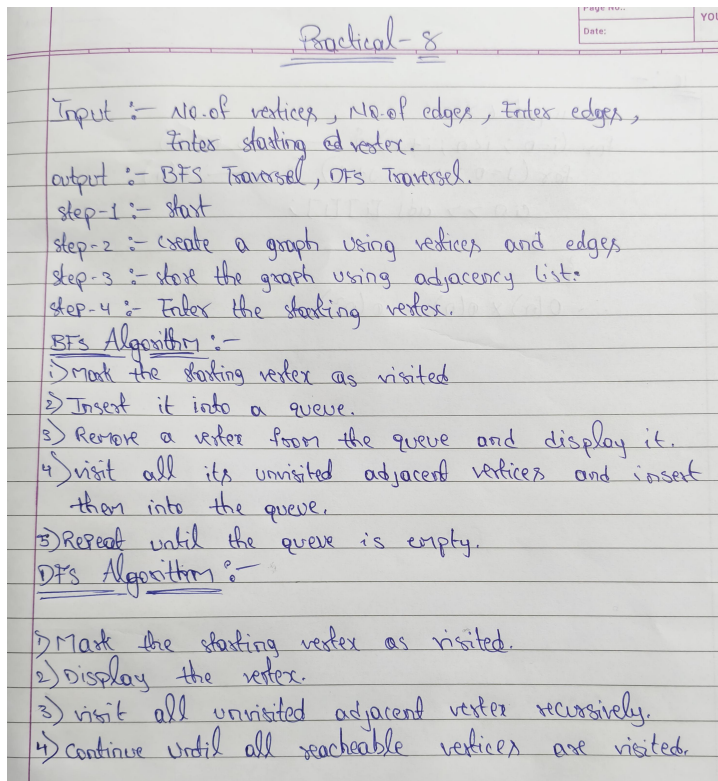


Practical –8

Aim: Implementation of graph and search (BFS & DFS)

Algorithm:



Program:

```
#include<iostream>
```

```
using namespace std;
```

```
class Graph {
```

```
public:
```

```
int adj [10][10];
```

```
int visited [10];
```

```
int n;
```

```
// — Get Input —
```

```
void getInput() {
```

```
    cout <<< "Enter number of vertices : ";
```

```
    cin >> n;
```

```
    cout <<< "Enter adjacency matrix (< n < x< < n < < ) :< < endl;
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < n; j++)
```

```
            cin >> adj[i][j];
```

```
}
```

```
// — Reset Visited —
```

```
void resetVisited() {
```

```
    for (int i = 0; i < n; i++)
```

```
        visited[i] = 0;
```

```
}
```

```
void BFS(int start) {
```

```
    int queue[10];
```

```
    int front = 0, rear = 0;
```

```
    resetVisited();
```

```
// enqueue start vertex

queue[rear++] = start;
visited[start] = 1;

cout <<< "BFS Traversal : ";

While (front < rear) {

    // dequeue
    int node = queue[front++];
    cout <<< node <<< " ";

    // visit all unvisited neighbours
    For (int i = 0; i < n; i++) {
        if(adj[node][i] == 1 & visited[i] == 0) {
            queue[rear++] = i;
            visited[i] = 1;
        }
    }
}

cout <<< endl;
}
```

```
void DFS(int node) {  
    visited[node] = 1;  
    cout <<< node <<< " ";  
  
    for(int i = 0; i < n; i++) {  
        if(adj[node][i] == 1 && visited[i] == 0) {  
            DFS(i); // recursive call  
        }  
    }  
}  
  
// — Start DFS —  
void startDFS(int start) {  
    resetVisited();  
    cout <<< "DFS Traversal : ";  
    DFS (start);  
    cout <<< endl;  
}  
  
// — Display Adjacency Matrix —  
void displayMatrix() {  
    cout <<< "Adjacency Matrix :";  
    cout <<< " ";  
    For (int i = 0; i < n; i++)  
        cout <<< i <<< " ";  
}
```

```
cout <<< endl;

for(int i = 0; i < n; i++) {
    cout <<< i <<< " ";

    for(int j = 0; j < n; j++)
        cout <<< adj[i][j] <<< " ";

    cout <<< endl;
}

};

// — MAIN —

int main () {

    Graph g ; // object

    g.getInput();
    g.displayMatrix();

    int start;

    cout <<< "Enter starting vertex : ";

    cin >> start;
```

```
cout <<< endl;  
  
g.BFS(start);  
  
g.startDFS(start);  
  
return 0;  
}
```

Output:

```
Enter number of vertices: 5  
Enter number of edges: 4  
Enter edges (u v):  
0 1  
0 2  
1 3  
2 4  
Enter starting vertex: 0  
  
DFS Traversal: 0 1 3 2 4  
  
BFS Traversal: 0 1 2 3 4  
  
Execution Time:  
DFS: 2593200 ns  
BFS: 727500 ns
```

Time complexity :

Tc :-

```
get input ()
for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
    for (j=0; j<n; j++) {  $\rightarrow O(n)$ 
         $G[i][j] > 0$  add [i][j];
    }
}
```

$\therefore O(n) \times O(n) = O(n^2)$

BFS:-

```
reset visited ()
for (i=0; i<n; i++)  $\rightarrow O(n)$ 
while (front < rear) {  $\rightarrow O(n)$ 
    for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
        ...
    }
}
```

Total Time complexity = $O(n) + O(n) \times O(n)$
= $O(n) + O(n^2)$
= $O(n^2)$

DFS:-

```
DFS (node)
for (i=0; i<n; i++) {  $\rightarrow O(n)$ 
    {
        if (.....)
            DFS (i);  $\rightarrow$  recursive call
    }
}
```

DFS visits at most n vertices $\rightarrow O(n)$

Therefore;

$$O(n) \times O(n) = O(n^2)$$

Time complexity (BFS) & (DFS) :-

Best case - $O(n^2)$:- The min no. matrix checks still take quadratic time.

Avg case - $O(n^2)$:- BFS & DFS process vertices and check their possible connections, resulting in quad time.

Worst case - $O(n^2)$:- All vertices are visited and all possible connections are checked, resulting in quadratic time.

Conclusion : Thus, the graph was successfully implemented using an adjacency matrix, and BFS and DFS traversals were performed successfully. The traversal methods helped in understanding how vertices of a graph can be visited systematically.